

LAPORAN MINGGU 6

Topik: Desain Arsitektur Sistem dengan UML dan Prinsip SOLID

Nama : Sri Wahyuningsih
Nim : 240202844
Kelas : 3IKRA

A. DESKRIPSI SINGKAT SISTEM

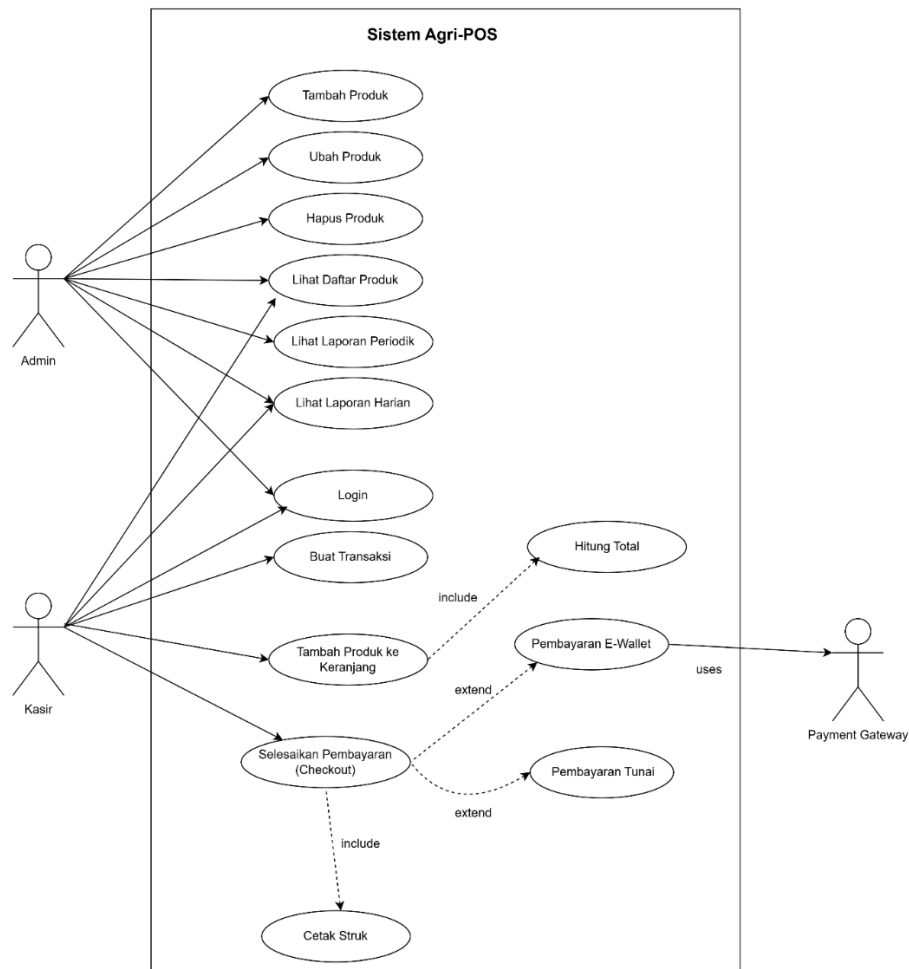
Sistem Agri-POS (Point of Sale Pertanian) adalah aplikasi kasir yang dirancang untuk toko atau usaha penjualan produk pertanian (benih, pupuk, alat, obat). Sistem ini bertujuan untuk memfasilitasi manajemen produk, transaksi penjualan tunai dan *e-wallet*, serta pencetakan struk dan pelaporan penjualan harian/periodik. Sistem membedakan hak akses antara Kasir (fokus pada transaksi) dan Admin (fokus pada manajemen produk dan laporan).

B. PENJELASAN DIAGRAM DAN KETERKAITANNYA

Desain sistem Agri-POS direpresentasikan melalui empat diagram UML utama untuk memetakan kebutuhan fungsional dan non-fungsional, memastikan struktur yang modular dan *maintainable*.

1. Use Case Diagram (UCD)

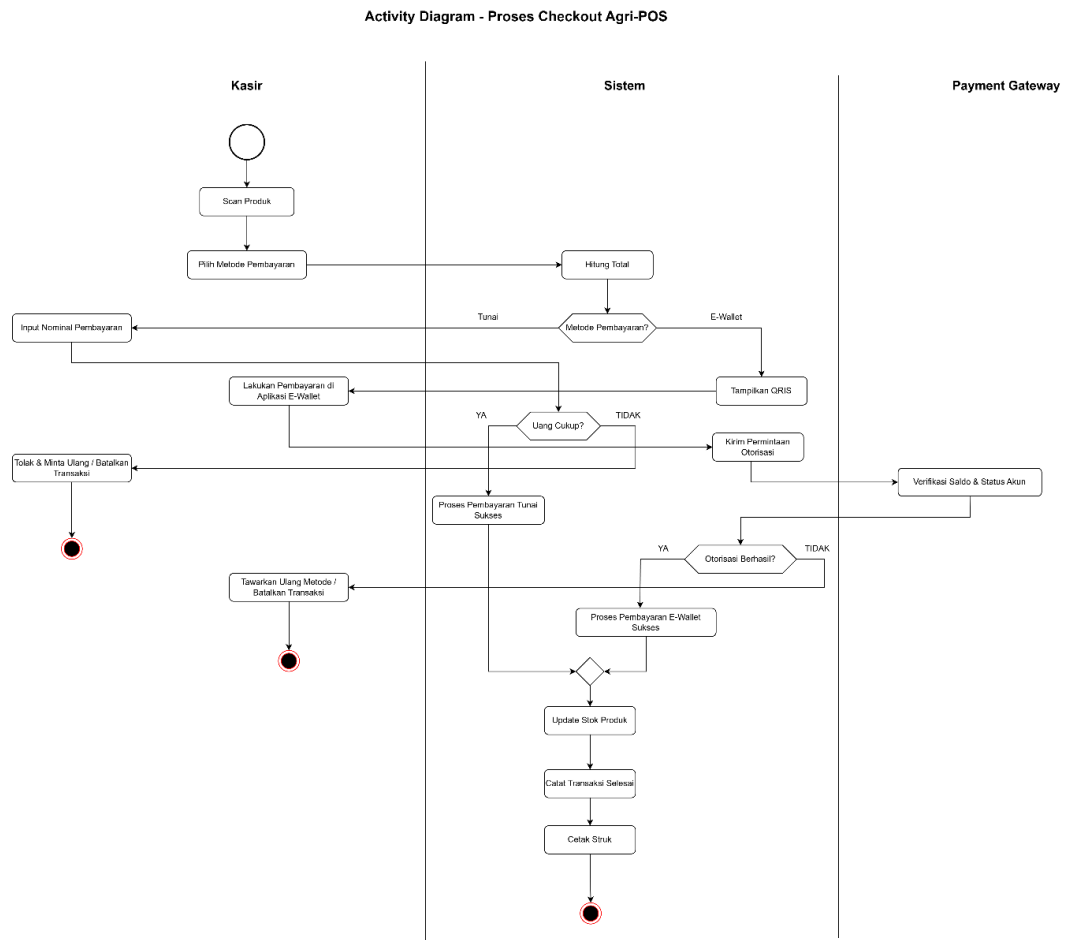
- Fungsi: Menjelaskan fungsionalitas sistem dari sudut pandang pengguna (aktor).
- Aktor: Admin (mengelola produk dan melihat laporan) dan Kasir (melakukan login, transaksi, dan pembayaran).
- Keterkaitan:
 - a) Proses Buat Transaksi dan Tambah Produk ke Keranjang mengarah ke Selesaikan Pembayaran (Checkout).
 - b) Selesaikan Pembayaran (Checkout) mencakup (*include*) Cetak Struk dan dapat diperluas (*extend*) menjadi Pembayaran Tunai atau Pembayaran E-Wallet.
 - c) Pembayaran E-Wallet berinteraksi (*uses*) dengan Payment Gateway.
 - d) Pembayaran Tunai menyertakan (*include*) Hitung Total.



2. Activity Diagram (ACD)

- Fungsi: Menggambarkan alur kerja proses utama sistem, yaitu proses Checkout dan Pembayaran.
- Swimlane: Dibagi menjadi tiga swimlane utama: Kasir, Sistem, dan Payment Gateway.
- Alur Utama: Kasir memulai dengan Scan Produk hingga Pilih Metode Pembayaran.
- Skenario Pembayaran:
 - a) Tunai: Sistem Hitung Total. Kasir Input Nominal Pembayaran. Sistem mengecek Uang Cukup?. Jika YA, proses Tunai Selesai. Jika TIDAK, menawarkan Tolak & Minta Uang / Batalkan Transaksi.
 - b) E-Wallet: Sistem menampilkan QRIS. Terjadi interaksi dengan Payment Gateway untuk Verifikasi Saldo & Status Akun. Sistem mengecek Otorisasi Berhasil?. Jika YA, proses E-Wallet Selesai. Jika TIDAK, menawarkan Tolak & Minta Ulang / Batalkan Transaksi.
- Akhir Proses: Kedua metode pembayaran yang berhasil akan dilanjutkan ke Update Stok Produk, Catat Transaksi Selesai, dan Cetak Struk.

- Keterkaitan: ACD memberikan detail operasional dari UC-Selesaikan Pembayaran (Checkout), termasuk bagaimana sistem menangani cabang logika (*decision node*) seperti pemilihan metode pembayaran dan penanganan alur sukses/gagal.



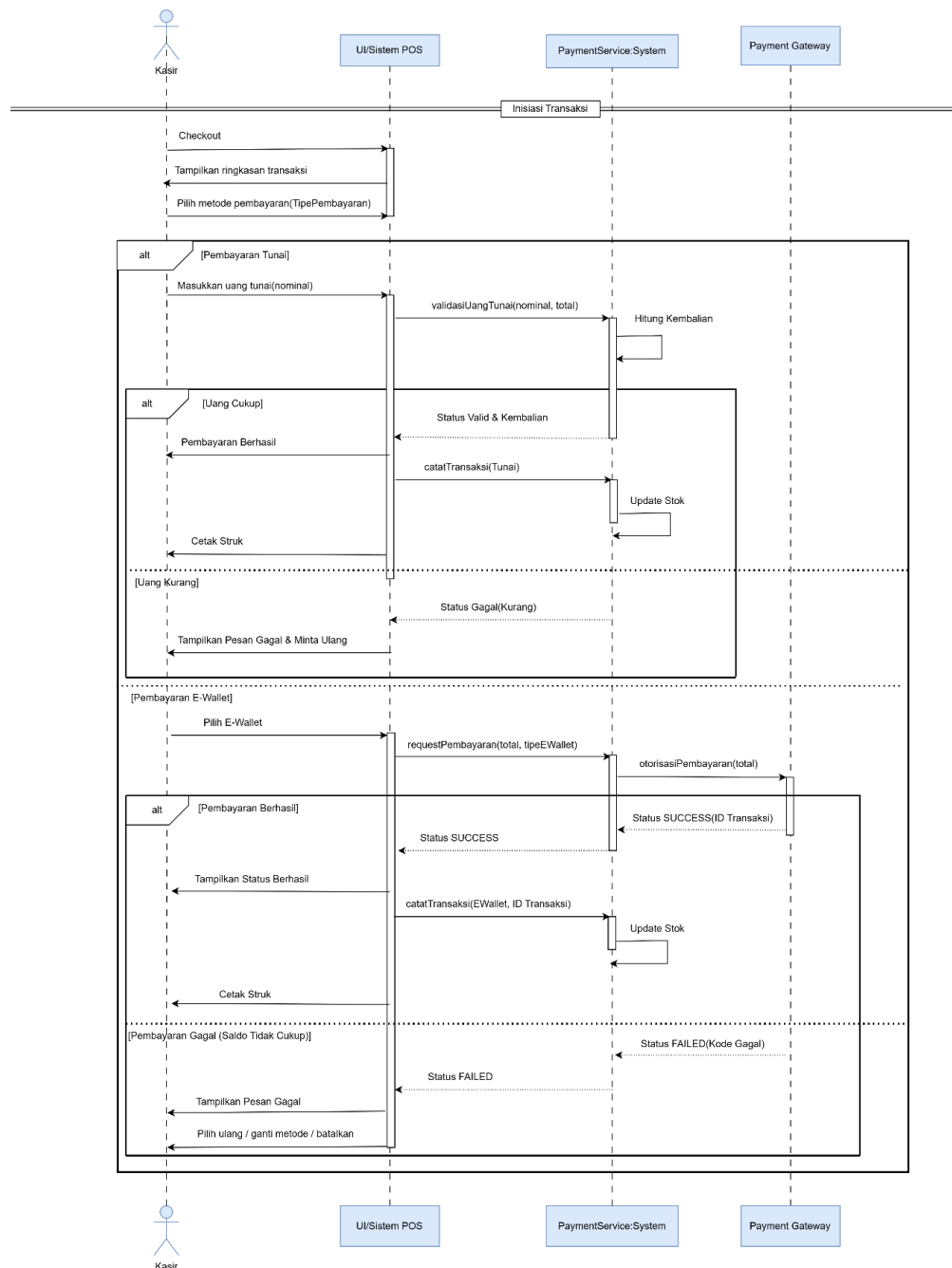
3. Sequence Diagram (SD)

- Fungsi: Menunjukkan urutan interaksi pesan antar objek/sistem selama proses Pembayaran Agri-POS.
- Lifeline: Melibatkan Kasir, UI/Sistem POS, PaymentService System, dan Payment Gateway.
- Skenario Alternatif (alt): Diagram memuat dua skenario pembayaran utama: Pembayaran Tunai dan Pilih E-Wallet.
- Error Handling:
 - a) Tunai: Terdapat *alt* untuk [Uang Kurang] yang menampilkan pesan gagal dan meminta ulang.

b) E-Wallet: Terdapat *alt* untuk [Pembayaran Gagal (Saldo Tidak Cukup)] yang menerima Status FAILED/Kode Gagal dari *Payment Gateway* dan menampilkan pesan gagal.

- Inti Proses: *PaymentService System* bertanggung jawab untuk validasi uang tunai, inisiasi transaksi, dan pembaruan stok. *Payment Gateway* digunakan khusus untuk otorisasi pembayaran E-Wallet.
- Keterkaitan: SD memberikan detail implementasi dari ACD Checkout. Misalnya, aktivitas Kirim Permintaan Otorisasi di ACD diimplementasikan sebagai pesan Request pembayaran yang dikirim dari UI/Sistem POS ke *Payment Gateway* di SD.

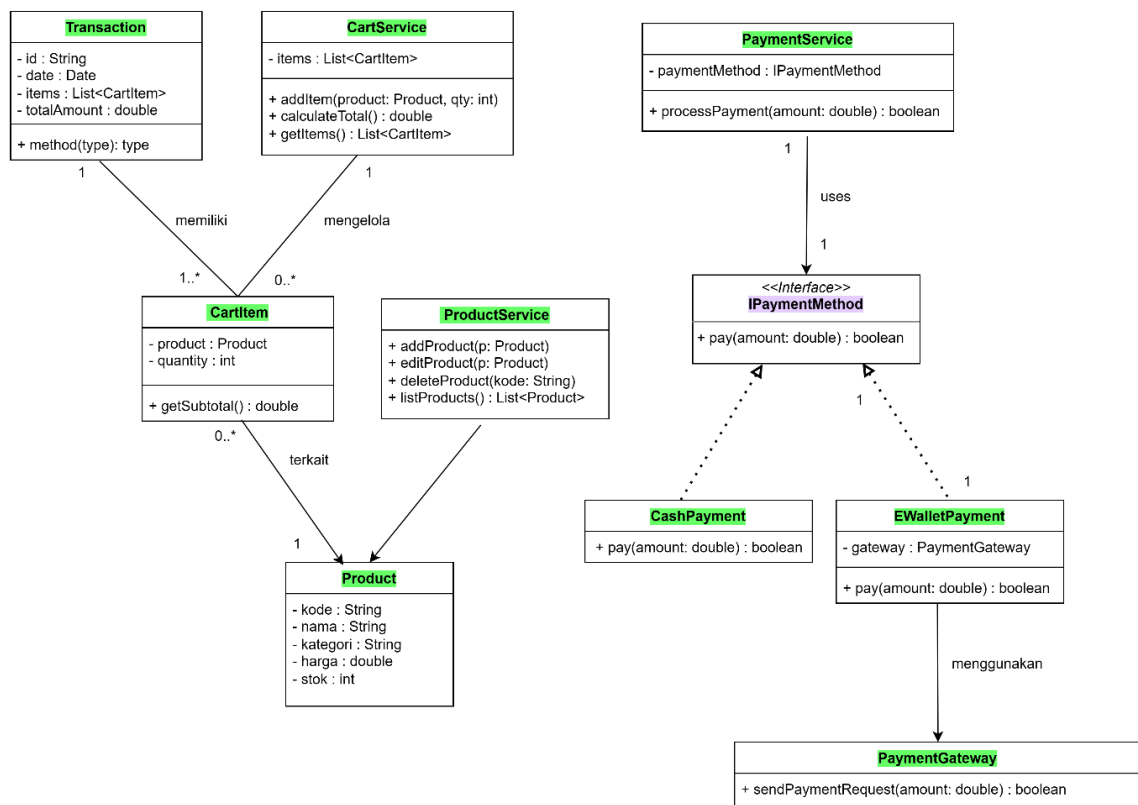
Sequence Diagram - Proses Pembayaran Agri-POS



4. Class Diagram (CD)

- Fungsi: Menjelaskan struktur statis sistem, class, atribut, operasi, dan relasi.
- Kelas Utama:
 - a) Product: Entitas data utama (kode, nama, kategori, harga, stok).
 - b) Transaction: Menyimpan detail transaksi (id, date, totalAmount).
 - c) CartItem: Detail produk dalam keranjang (product, quantity).
 - d) ProductService: Logika bisnis manajemen produk (addProduct, editProduct, deleteProduct).
 - e) CartService: Logika keranjang (addItem, calculateTotal).
 - f) PaymentService: Mengelola proses pembayaran.
- Relasi Penting:
 - a) Inheritance/Pewarisan: CashPayment dan EWalletPayment mewarisi dari *Interface* IPaymentMethod. (Penerapan SOLID)
 - b) Asosiasi/Multiplicity: *Transaction* memiliki 1..* *CartItem*. *CartItem* terkait dengan 1 *Product*. *PaymentService* menggunakan 1 *IPaymentMethod*.
- Keterkaitan: Kelas-kelas di CD (misalnya, PaymentService, Transaction) adalah objek yang berinteraksi dalam SD dan menjalankan fungsi yang dideskripsikan dalam ACD dan UCD.

Class Diagram - Sistem Agri-POS



C. PENJELASAN PENERAPAN PRINSIP SOLID

Desain arsitektur Agri-POS secara eksplisit menerapkan beberapa prinsip SOLID untuk menjamin *maintainability* dan *extensibility* sistem, khususnya pada modul pembayaran.

1) Single Responsibility Principle (SRP)

- Penerapan: Setiap kelas dirancang untuk hanya memiliki satu alasan untuk berubah.
 - a) ProductService: Bertanggung jawab tunggal hanya untuk manajemen data produk (tambah, ubah, hapus produk).
 - b) CartService: Bertanggung jawab tunggal hanya untuk manajemen keranjang (tambah item, hitung total).
 - c) PaymentService: Bertanggung jawab tunggal hanya untuk mengorkestrasi proses pembayaran dengan memanggil implementasi *IPaymentMethod*.
 - d) Transaction: Bertanggung jawab tunggal untuk menyimpan status transaksi yang telah selesai.
- Tidak ada kelas yang bertanggung jawab ganda (misalnya, Product tidak menangani manajemen stok DAN pembayaran).

2) Open/Closed Principle (OCP)

- Penerapan: Entitas harus terbuka untuk perluasan, tetapi tertutup untuk modifikasi.
 - a) Modul Pembayaran: Kelas inti PaymentService menerima IPaymentMethod sebagai dependensi.
 - b) Perluasan: Ketika metode pembayaran baru seperti *TransferBankPayment* akan ditambahkan, kita cukup membuat kelas baru yang mengimplementasikan IPaymentMethod.
 - c) Tertutup Modifikasi: Kelas PaymentService tidak perlu diubah sama sekali untuk mendukung metode pembayaran yang baru.

3) Liskov Substitution Principle (LSP)

- Penerapan: Objek dalam program harus dapat diganti dengan *sub-tipe* mereka tanpa merusak fungsionalitas program.
 - a) Modul Pembayaran: Kelas CashPayment dan EWalletPayment adalah *sub-tipe* dari IPaymentMethod.
 - b) Penggantian: PaymentService dapat menggunakan objek CashPayment atau EWalletPayment secara bergantian melalui *interface* IPaymentMethod tanpa mengetahui implementasi spesifik di dalamnya. Metode pay(amount: double) akan selalu berfungsi sesuai kontrak yang didefinisikan oleh *interface*.

4) Interface Segregation Principle (ISP)

- Definisi: Klien (kelas yang menggunakan *interface*) tidak boleh dipaksa untuk bergantung pada *method* yang tidak mereka gunakan. Lebih baik menggunakan banyak *interface* kecil daripada satu *interface* besar.
- Penerapan: Diagram ini fokus pada *interface* kecil yang relevan. *IPaymentMethod* hanya berisi satu *method* (*pay*), yang relevan untuk semua implementasi pembayaran. Jika ada *method* yang hanya relevan untuk E-Wallet (misalnya, *validateToken*), *method* tersebut akan dipisahkan ke *interface* lain (misalnya, *IEWalletValidation*) yang hanya diimplementasikan oleh *EWalletPayment*, sehingga *CashPayment* tidak dipaksa untuk mengimplementasikannya.

5) Dependency Inversion Principle (DIP)

- Penerapan: Modul tingkat tinggi harus bergantung pada abstraksi, bukan pada detail konkret.
 - a) Kelas tingkat tinggi *PaymentService* bergantung pada *IPaymentMethod* (abstraksi).
 - b) Ketergantungan ini memungkinkan *injecting* implementasi yang berbeda (*cash*, *e-wallet*, atau bahkan *mock* untuk pengujian).
 - c) Ini secara drastis meningkatkan Testability, karena saat menguji *PaymentService*, kita bisa menyuntikkan objek *mock* (palsu) dari *IPaymentMethod* tanpa perlu koneksi ke *PaymentGateway* yang sebenarnya.

TABEL TRACEABILITY (FR → DIAGRAM → KELAS/INTERFACE REALISASI)

Tabel ini menunjukkan konsistensi dan pemetaan antara kebutuhan fungsional (FR) dan elemen perancangan:

FR	Use Case	Activity/Sequence	Class/Interface
Manajemen Produk (Tambah, Ubah, Hapus, Tampil)	UC-Tambah Produk, UC-Ubah Produk, UC-Hapus Produk, UC-Lihat Daftar Produk	Tidak Terlibat	<i>ProductService</i> , <i>Product</i>
Transaksi Penjualan (Buat, Tambah Keranjang, Hitung Total)	UC-Buat Transaksi, UC-Tambah Produk ke Keranjang, UC-Hitung Total	Seq Pembayaran	<i>CartService</i> , <i>CartItem</i> , <i>Transaction</i>
Perhitungan Total	UC-Hitung Total	Activity/Sequence Transaksi	<i>CartService</i>

Metode Pembayaran (Tunai dan E-Wallet)	UC-Selesaikan Pembayaran (Checkout), UC- Pembayaran Tunai, UC-Pembayaran E- Wallet	Activity Checkout, Seq Pembayaran (Cash/EWallet)	IPaymentMethod, CashPayment, EWalletPayment, PaymentService
Pencetakan Struk	UC-Cetak Struk	Activity Checkout, Seq Pembayaran	Transaction
Laporan	UC-Lihat Laporan Harian, UC-Lihat Laporan Periodik	Tidak Terlibat	Transaction

D. KESIMPULAN DAN REFLEKSI SINGKAT

Keunggulan Sistem

- Ekstensibilitas Tinggi (OCP & DIP): Desain pembayaran yang menggunakan *interface* *IPaymentMethod* memastikan penambahan metode pembayaran baru di masa depan tidak memerlukan perubahan pada logika *PaymentService* yang sudah ada.
- Modularitas dan *Maintainability* (SRP): Pemisahan tanggung jawab ke dalam *Service Class* (*ProductService*, *CartService*, *PaymentService*) membuat kode lebih terorganisir, mudah diuji, dan lebih *maintainable*.
- Keterlacakan (Traceability): Penggunaan minimal empat diagram UML yang saling konsisten memfasilitasi keterlacakan setiap *Functional Requirement* ke komponen desain (kelas/interface) yang merealisasikannya.

Potensi Pengembangan Sistem

- Implementasi Pola Strategy: Untuk *Next-Level Requirement* Diskon/Promosi, pola *Strategy* dapat diterapkan (mirip dengan *IPaymentMethod*) untuk memungkinkan penambahan aturan diskon yang fleksibel tanpa mengubah kelas inti *Transaction* atau *CartService*.
- Repository Pattern: Untuk meningkatkan *testability* dan *portability*, dapat ditambahkan *interface* *IProductRepository* (sesuai *Guidance SOLID*), dan *ProductService* akan bergantung pada *interface* ini (DIP).

JAWABAN QUIZ (SESUAI RPS)

1. Jelaskan perbedaan *aggregation* dan *composition* serta berikan contoh penerapannya pada desain Anda.

Jawab:

Pembeda	Aggregation (Asosiasi Lemah)	Composition (Asosiasi Kuat/Kepemilikan)
Ketergantungan Hidup	Opsional. Objek <i>part</i> dapat berdiri sendiri ketika objek <i>whole</i> dihancurkan.	Wajib. Objek <i>part</i> akan ikut hancur jika objek <i>whole</i> dihancurkan.
Notasi UML	Garis dengan berlian kosong di sisi <i>whole</i> .	Garis dengan berlian terisi solid di sisi <i>whole</i> .
Contoh Aplikasi	ProductService <-> Product. Product tetap ada di database meskipun ProductService dimatikan.	Tidak ada komposisi kuat yang eksplisit pada diagram ini, tetapi contoh umum adalah Transaction <-> CartItem (jika CartItem hanya ada sebagai bagian dari Transaction dan tidak dapat eksis tanpanya). Dalam konteks ini, kami menggunakan Asosiasi Biasa (implisit <i>composition</i> jika di-code) untuk Transaction dan CartItem karena CartItem adalah detail transaksi yang harus ikut terhapus bersama transaksinya.

2. Bagaimana prinsip *Open/Closed* dapat memastikan sistem mudah dikembangkan?

Jawab:

Prinsip Open/Closed Principle (OCP) memastikan sistem mudah dikembangkan dengan cara memisahkan bagian yang cenderung berubah (variasi/detail) dari bagian yang harus stabil (inti/abstraksi).

- Pengembangan (Ekstensi): Ketika fungsionalitas baru (misalnya, metode pembayaran baru) diperlukan, pengembang hanya menambahkan kelas baru yang mengimplementasikan *interface* (IPaymentMethod).

- Stabilitas (Modifikasi): Kode inti (seperti `PaymentService`) yang telah teruji dan stabil tidak perlu diubah sama sekali. Ini mengurangi risiko *bug* pada kode yang sudah ada dan mempercepat proses pengembangan.

3. Mengapa *Dependency Inversion Principle (DIP)* meningkatkan *testability*? Berikan contoh penerapannya.

Jawab:

DIP meningkatkan *testability* (kemampuan uji) karena menghilangkan ketergantungan langsung (*hard coupling*) pada implementasi konkret.

- Peningkatan *Testability*: Dengan DIP, kita dapat dengan mudah mengganti implementasi konkret dari suatu *dependency* dengan versi palsu (Mock atau Stub) saat pengujian.
- Contoh Penerapan pada Agri-POS:
 - Kelas `PaymentService` tidak bergantung pada `EWalletPayment` (implementasi konkret), melainkan pada `IPaymentMethod` (abstraksi).
 - Saat menguji `PaymentService`, kita bisa menyuntikkan (melalui *Constructor Injection*) `MockPaymentMethod` alih-alih `EWalletPayment` yang nyata. *Mock* ini dapat diprogram untuk selalu mengembalikan SUCCESS atau FAILED tanpa perlu terhubung ke *Payment Gateway* yang sebenarnya, sehingga pengujian unit menjadi cepat, terisolasi, dan terpercaya.

CHECKLIST PEMERIKSAAN MANDIRI

- ✓ Semua FR tercover di Use Case Diagram
- ✓ Activity & Sequence memuat alur sukses dan gagal (alt/opt)
- ✓ Class Diagram memiliki visibility, tipe, multiplicity, dan paket
- ✓ Mapping SOLID (min 3) ditunjukkan di desain dan dijelaskan di laporan
- ✓ Konsistensi penamaan antar diagram terjaga
- ✓ File sumber diagram disertakan (src/uml/) + gambar docs/
- ✓ Commit mengikuti format week6-uml-solid: iterasi-N <deskripsi>